

B103B Database & Big Data – Database Project Report

GH1042431 – Phuc Lam Nguyen

GitHub URL: <https://github.com/LamGH1042431/B103B-Database-Project>

Video URL: [Screen Recording 2026-06-26 at 14.19.32.mov](#)

Introduction

Domain

My database is based on a hotel reservation system. I chose to work in this domain as it had clear entities that are easy to visualize. For instance, a customer makes a booking, a booking includes a room, and a booking has a payment. Compared to a banking system for instance it may require trickier business logic and E-commerce usually require many tables.

Goals of the System

The database consists of four entities. Firstly, the Customer. Creating a customer's table will help with storing contact information whenever a customer books or reserves a room. Secondly, the Payment. This should help track what kind of payment method was made, the amount that is due and even the payment date and status. Thirdly, the Reservation. This table should show the booking date, the check in and check out date as well as the reservation status of the customers booking. Lastly, the Room. This should include the room numbers; room type as well as the price per night and status of a room. The goal of the system is to efficiently manage room availability, track customer payments, and process booking.

System Design

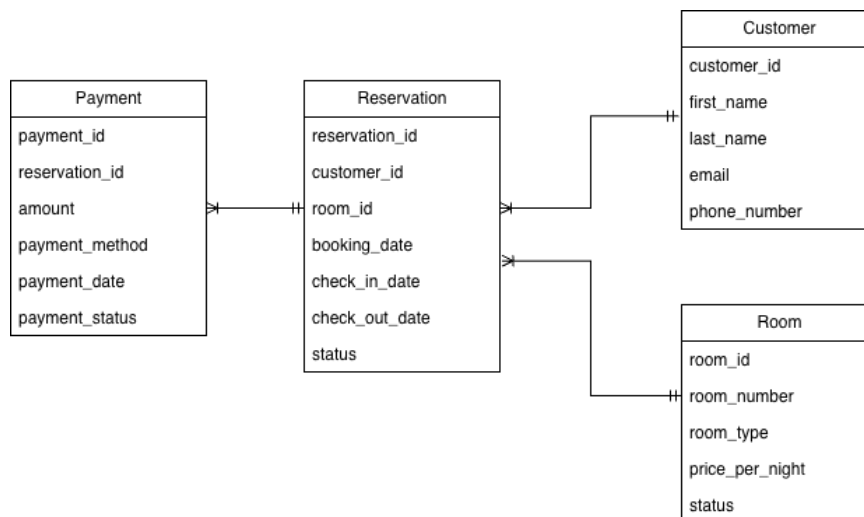


Figure 1: ER Diagram of Hotel Reservation System

Relationships

Figure 1 illustrates an ER diagram of my system, and the notations explain what relationship each entity has with each other. Reservation to payment is a one-to-many relationship since a customer can pay a deposit when booking and pay the remaining balance on check out or a payment fails and they try again with a different method.

Customer to reservation is a one-to-many relationship since a customer can make multiple bookings for different rooms. Another example is that a customer can stay at the hotel in January then come again in February. Each visit is a separate reservation. This means the customer has two reservations over time. Room to reservation is also a one-to-many relationship since different customers can book the same room on separate dates so the same room can have many reservations across different time periods.

My Tables

A hotel reservation system usually requires customer contact since the hotel needs to know who is making the booking. A room table is required because the hotel needs to know what is being booked. A reservation table is needed to record the booking itself, and the payment table is needed to track how a customer pays. In a real-world scenario, a hotel system would not work without these tables. That is why I chose to do these tables.

Implementation

Customer Schema

Figure 2 illustrates the schema for my customer's table. In the customer's table, it is required to have a customer ID, a first and last name, as well as the email and phone number to track customer contact when they make a reservation. To create a table for customers, using a 'CREATE TABLE,' a Data Definition Language (DDL) is used along with the table name, "Customer."

```
CREATE TABLE Customer (  
  customer_id INT AUTO_INCREMENT PRIMARY KEY,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  email VARCHAR(50) NOT NULL UNIQUE,  
  phone_number VARCHAR(20) NOT NULL  
);
```

Figure 2: Schema for Customer Table

1. Customer ID

In every table, it requires a primary key as it helps to uniquely identify each row in a table. No two rows can have the same primary key value. This is how other tables reference this table via foreign key. In this case, I assigned the constraint for 'customer_id' as the 'PRIMARY KEY'.

The data type for the customer ID is set to 'INT' since it is an integer and is usually a set of numbers. Assigning the 'AUTO_INCREMENT' function lets MySQL to handle it automatically without having to manually assign IDs. Not using this function would risk of duplicates or missing IDs and means I will not have to track which IDs have been used. It is convenient, reliable, and simple.

2. First and Last Name

In the customer's table, the first and last name is assigned to 'VARCHAR' because names are often texts of varying lengths. Setting this data type to a maximum of 50 characters seemed generous enough for any realistic name. I have also set the constraint to 'NOT NULL' since in a real case scenario, a first and last name cannot be empty when a customer makes a booking. So 'NOT NULL' ensures that a customer record can never be created without a name.

3. Email

Email is set to 'VARCHAR(50)' since it is also a text of varying lengths. However, I assigned the email with two constraints, 'NOT NULL,' and 'UNIQUE.' It is 'NOT NULL' because like the customer's name, it cannot be empty when a customer makes a booking, and emails are always unique from each other meaning two or more people cannot have the same email, that is why it is set to 'UNIQUE.'

4. Phone Number

This is set to 'VARCHAR' and not 'INT' because a phone number is not only a set of numbers. For instance, +491511234567 is a German number and if it is stored as 'INT,' the '+' would be dropped or cause an error. Setting the data type to maximum of 20 characters seemed sensible and reasonable for a phone number.

Room Schema

A table is created by using 'CREATE TABLE' with the table name, "Room." In the room's table, the attributes are as followed, room ID, room number, room type, the price per night and the status of the room.

```
CREATE TABLE Room (  
  room_id INT AUTO_INCREMENT PRIMARY KEY,  
  room_number INT UNIQUE NOT NULL,  
  room_type VARCHAR(50) NOT NULL,  
  price_per_night DECIMAL (10,2) NOT NULL,  
  status ENUM ('available', 'occupied', 'maintenance') DEFAULT 'available'  
);
```

Figure 3: Schema for Room Table

1. Room ID

Like any other table, figure 3 needs a primary key. In this case room ID is assigned as the primary key. The data type is then set to 'INT' since ID is an integer. Setting the column attribute to 'AUTO_INCREMENT' will remove any manual assigning in the future.

2. Room Number

The room number is an integer so setting it to 'INT' would be appropriate. Room numbers should also be unique since two or more rooms cannot be the same. A room can also not be empty, so the constraints are set to 'UNIQUE' and 'NOT NULL.'

3. Room Type

A hotel can have different types of rooms. For example, a single room, a double room, or a suite. It is reasonable to set the data type to 'VARCHAR,' but setting the maximum characters of 50 is generous in this case. However, it would be more appropriate to set it to 'ENUM' when there are only three sets of rooms. For example, "room_id ENUM ('single', 'double', 'suite') NOT NULL." A room cannot have no room type, so it is necessary to set the constraint to 'NOT NULL.'

4. Price Per Night

The price of something is always in decimals so it is appropriate to use 'DECIMAL(10,2)' as a data type. As prices can vary, small or large, it seemed reasonable to set the precision and scale to (10,2). This ensures accurate storage of currency with two decimal places to avoid rounding errors.

5. Status of Room

Status is set to 'ENUM' since it should only have a fixed set of values. In this case, I have set it to 'available,' 'occupied,' and 'maintenance.' Assigning the 'DEFAULT' constraint to 'available' ensures that if no value is provided for a column, the predefined value is used automatically. If a room is not booked it should be available by default.

Reservation Schema

```
CREATE TABLE Reservation (  
  reservation_id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  room_id INT NOT NULL,  
  booking_date DATE NOT NULL,  
  check_in_date DATE NOT NULL,  
  check_out_date DATE NOT NULL,  
  status ENUM ('reserved', 'checked_in', 'checked_out', 'cancelled') DEFAULT 'reserved',  
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),  
  FOREIGN KEY (room_id) REFERENCES Room(room_id)  
);
```

Figure 4: Schema for Reservation Table

1. Reservation, Customer, & Room ID

Since figure 4 is a schema for the reservation's table, the reservation ID is assigned as the primary key and is auto incremented. When a customer makes a reservation, it should link to who is making the booking and what room is being booked so it would make sense to include the customer and room ID in the schema. They are not auto incremented since they reference existing IDs from other tables. All IDs should be set to 'INT' as the data type and 'NOT NULL' since they already exist in other tables and cannot be empty.

2. Booking, Check In, & Check Out Date

A hotel needs the date of when the booking was first reserved as well as the check in and check out date of the customer. Since these attributes are dates, the data type 'DATE' is used and are set to 'NOT NULL' as a reservation cannot exist without these dates.

3. Status of Reservation

'ENUM' is used in figure 4 to input a fixed set of values. The values are 'reserved,' 'checked_in,' 'checked_out,' and 'cancelled.' The 'DEFAULT' constraint is set to 'reserved' because when a customer makes a booking the status at that moment would be reserved. So naturally, the starting point of every booking should be reserved.

4. Foreign Key

As customer and room ID are included in the schema, two foreign key constraints should be made for them. This ensures that no reservation can be created without a valid customer and room. They help maintain referential integrity.

Payment Schema

```
CREATE TABLE Payment (
  payment_id INT AUTO_INCREMENT PRIMARY KEY,
  reservation_id INT NOT NULL,
  amount DECIMAL (10,2) NOT NULL,
  payment_method ENUM ('card', 'cash', 'transfer') NOT NULL,
  payment_date DATE NOT NULL,
  payment_status ENUM ('approved', 'declined', 'pending') DEFAULT 'pending',
  FOREIGN KEY (reservation_id) REFERENCES Reservation(reservation_id)
);
```

Figure 5: Schema for Payment Table

1. Payment & Reservation ID

Payment ID is the unique identifier in figure 5. When a payment is made at a hotel, the hotel needs to know which reservation that payment is for. That is why reservation ID is included in the schema and set as 'INT' and 'NOT NULL.'

2. Amount

Amount is the price of the payment and is then set to as 'DECIMAL (10,2)' and 'NOT NULL' since a booking needs a price. Prices can range from twenty euros to a thousand, so the precision and scale is set to (10,2).

3. Payment Method

Payment method is set to 'ENUM' and the values are 'card,' 'cash,' and 'transfer.' A transfer for instance can be through a bank or a wire transfer. All three are common payment methods for hotels.

4. Payment Date

Payment date is needed to keep record of when a payment is made. This is assigned as 'DATE' and 'NOT NULL.'

5. Payment Status

Payment status is set to 'ENUM' and the values are set to 'approved,' 'declined,' and 'pending.' The 'DEFAULT' constraint is set to 'pending.' A payment should naturally be pending if it has not been approved or declined.

Data Insertion

In figure 6, 8, 10, and 12, a Data Manipulation Language (DML) is used to execute the following data tables.

'INSERT INTO' is used, followed up with the table name and attributes. Doing so, I can insert the values to create each data table. See in figure 7, 9, 11, and 13 after it has been executed.

NOTE: The status columns in both the reservation and payment table in figures 11 and 13 were designed to reflect the natural life cycle of a hotel booking. For example, a new reservation as reserved will have a pending payment. On check out the payment is updated to approved. If a booking is cancelled, the payment is declined. Same applies for the room status in figure 9. If the room is available, the reservation status should be checked out and payment status should either be approved or pending. The status columns should make sense in reference to each other.

1. Data Insertion & Table for Customer

```
➤ INSERT INTO Customer (first_name, last_name, email, phone_number)
VALUES
('Maria', 'Schmidt', 'maria.schmidt@email.com', '+491511234567'),
('James', 'Wilson', 'james.wilson@email.com', '+491522345678'),
('Sofia', 'Müller', 'sofia.mueller@email.com', '+491533456789'),
('Lucas', 'Anderson', 'lucas.anderson@gmail.com', '+491544567890'),
('David', 'Nguyen', 'david.nguyen@gmail.com', '+491510123456');
```

Figure 6: Customer Table Data Insertion

customer_id	first_name	last_name	email	phone_number
1	Maria	Schmidt	maria.schmidt@email.com	+491511234567
2	James	Wilson	james.wilson@email.com	+491522345678
3	Sofia	Müller	sofia.mueller@email.com	+491533456789
4	Lucas	Anderson	lucas.anderson@gmail.com	+491544567890
5	David	Nguyen	david.nguyen@gmail.com	+491510123456

Figure 7: Customer Table

2. Data Insertion & Table for Room

```
➤ INSERT INTO Room (room_number, room_type, price_per_night, status)
VALUES
(101, 'single', 80.00, 'available'),
(102, 'double', 120.00, 'available'),
(103, 'suite', 250.00, 'available'),
(104, 'double', 130.00, 'occupied'),
(105, 'single', 85.00, 'maintenance');
```

Figure 8: Room Table Data Insertion

123 room_id	123 room_number	AZ room_type	123 price_per_night	AZ status
1	101	single	80	available
2	102	double	120	available
3	103	suite	250	available
4	104	double	130	occupied
5	105	single	85	maintenance

Figure 9: Room Table

3. Data Insertion & Table for Reservation

```

INSERT INTO Reservation (customer_id, room_id, booking_date, check_in_date, check_out_date, status)
VALUES
(1, 1, '2025-01-10', '2025-01-15', '2025-01-20', 'checked_out'),
(2, 2, '2025-02-01', '2025-02-05', '2025-02-10', 'reserved'),
(3, 3, '2025-03-01', '2025-03-10', '2025-03-15', 'cancelled'),
(4, 4, '2025-04-01', '2025-04-05', '2025-04-08', 'checked_in'),
(5, 5, '2025-05-01', '2025-05-10', '2025-05-15', 'cancelled');

```

Figure 10: Reservation Table Data Insertion

123 reservation_id	123 customer_id	123 room_id	booking_date	check_in_date	check_out_date	AZ status
1	1	1	2025-01-10	2025-01-15	2025-01-20	checked_out
2	2	2	2025-02-01	2025-02-05	2025-02-10	reserved
3	3	3	2025-03-01	2025-03-10	2025-03-15	cancelled
4	4	4	2025-04-01	2025-04-05	2025-04-08	checked_in
5	5	5	2025-05-01	2025-05-10	2025-05-15	cancelled

Figure 11: Reservation Table

4. Data Insertion & Table for Payment

```

INSERT INTO Payment (reservation_id, amount, payment_method, payment_date, payment_status)
VALUES
(1, 400.00, 'cash', '2025-01-20', 'approved'),
(2, 600.00, 'transfer', '2025-02-01', 'pending'),
(3, 1250.00, 'card', '2025-03-01', 'declined'),
(4, 390.00, 'card', '2025-04-05', 'pending'),
(5, 425.00, 'cash', '2025-05-01', 'declined');

```

Figure 12: Payment Table Data Insertion

123 payment_id	123 reservation_id	123 amount	AZ payment_method	payment_date	AZ payment_status
1	1	400	cash	2025-01-20	approved
2	2	600	transfer	2025-02-01	pending
3	3	1,250	card	2025-03-01	declined
4	4	390	card	2025-04-05	pending
5	5	425	cash	2025-05-01	declined

Figure 13: Payment Table

Queries & Results

1. Selecting all Available Rooms

Figure 14 illustrates how to execute and show all available rooms by using the 'SELECT' clause with '*' to select everything from a table. Using the 'FROM' clause with the table name and "WHERE status = 'available';" will execute and show all available rooms. See in figure 15. However, this would cause a foreign key error.

NOTE: See under "Challenges & Solutions" for the solution.

```
SELECT *  
FROM Room  
WHERE status = 'available';
```

Figure 14: Query for Selecting Available Rooms

123 room_id	123 room_number	A-Z room_type	123 price_per_night	A-Z status
1	101	single	80	available
2	102	double	120	available
3	103	suite	250	available

Figure 15: Table Showing Available Rooms

2. Selecting Active Reservations with Customer Names

Figure 16 demonstrates how to show and join active reservations with customer names from two different tables. First, the customer ID, and customer name is selected as well as the reservation ID and status. By selecting these attributes from the customer and reservation table will help with referencing and combining the two tables together using the 'JOIN' clause. Then to set the condition using 'WHERE,' the values 'reserved' and 'checked in' are inputted. See in figure 17 after query has been executed.

```
SELECT C.customer_id, C.first_name, C.last_name, R.reservation_id, R.status  
FROM Customer C  
JOIN Reservation R ON C.customer_id = R.customer_id  
WHERE R.status IN ('reserved', 'checked_in');
```

Figure 16: Query for Active Reservations with Customer Names

123 customer_id	A-Z first_name	A-Z last_name	123 reservation_id	A-Z status
2	James	Wilson	2	checked_in
4	Lucas	Anderson	4	checked_in

Figure 17: Joint Table for Active Reservations

3. Selecting Total Payments Approved

'SUM()' is an aggregated function that is used in figure 18 to add up all the approved payment amounts to give one single total number. Using 'AS' to change the column name to "total." Setting the condition for the payment status to approved is done by

using the 'WHERE' clause. Once the query is executed, you will see the result in figure 19.

```
SELECT SUM(amount) AS total
FROM Payment P
WHERE P.payment_status = 'approved';
```

Figure 18: Query for Total Payments Approved

123 total
400

Figure 19: Table with Total Payments Approved

4. Selecting All Cancelled Reservations

Like selecting all available rooms in figure 14, the available status in "WHERE status = 'available';" is changed to 'cancelled.' Using "FROM Reservation R" instead of 'Room' to select the table from the reservation status. See in figure 21.

```
SELECT *
FROM Reservation R
WHERE R.status = 'cancelled';
```

Figure 20: Query for Cancelled Reservations

reservation_id	customer_id	room_id	booking_date	check_in_date	check_out_date	status
3	3	3	2025-03-01	2025-03-10	2025-03-15	cancelled
5	5	5	2025-05-01	2025-05-10	2025-05-15	cancelled

Figure 21: Table for Cancelled Reservations

5. Updating Reservation Status to Checked In

In this query, to update the reservation status to checked in, the 'UPDATE' statement is used alongside the 'SET' clause to modify an existing record in the reservation table. The 'SET' clause changes the status column to 'checked_in' while the 'WHERE' clause ensures only the reservation with a reservation ID of 2 is updated. This prevents all other records from being updated since reservation ID 2 is the only one put in as reserved. Find the updated table the status column for reservation ID 2 in figure 23 compared to figure 11.

```
UPDATE Reservation
SET status = 'checked_in'
WHERE reservation_id = 2;
```

Figure 22: Query for Updating Reservation Status

123 reservation_id	123 customer_id	123 room_id	booking_date	check_in_date	check_out_date	AZ status
1	1	1	2025-01-10	2025-01-15	2025-01-20	checked_out
2	2	2	2025-02-01	2025-02-05	2025-02-10	checked_in
3	3	3	2025-03-01	2025-03-10	2025-03-15	cancelled
4	4	4	2025-04-01	2025-04-05	2025-04-08	checked_in
5	5	5	2025-05-01	2025-05-10	2025-05-15	cancelled

Figure 23: Updated Reservation Table

6. Removing a Customer from the Data Table

In one valid scenario, deleting a customer can happen when a payment is declined or when they never completed their booking. Using 'DELETE FROM' with the table name alongside "WHERE customer_id = 5;" will delete this specific customer with a payment status that is declined in the payment table. See in figure 7 and 13 for reference.

```
DELETE FROM Customer
WHERE customer_id = 5;
```

Figure 24: Removing a Customer

7. Finding the Total Revenue of Rooms that are Approved

By selecting room type with the aggregation 'SUM()' function where the column reference is amount in the payment table alongside 'AS' to create an alias as total revenue to the result. We want to use "FROM Payment P" since the payment status is on this table. Two 'JOIN' clauses are then used to join the reservation table with the room and payment table. Since the room table has no foreign key in the payment table, the reservation table is used to join these tables together as room and payment ID are both in this table. We leave the condition where the payment status is approved using the 'WHERE' clause. The 'GROUP BY' clause groups the results by the room type so that the 'SUM()' aggregation function calculates the total revenue separately for each room type, rather than returning one combined total. See in figure 26 for the result.

```
SELECT R.room_type, SUM(P.amount) AS total_revenue
FROM Payment P
JOIN Reservation RE ON P.reservation_id = RE.reservation_id
JOIN Room R ON RE.room_id = R.room_id
WHERE P.payment_status = 'approved'
GROUP BY R.room_type
```

Figure 25: Query for Total Revenue of Room that has been Approved

A-Z room_type	123 total_revenue
single	400

Figure 26: Result Table for Total Revenue that has been Approved

Challenges & Solutions

The first challenge I faced were the data type decisions I had to make. For example, in figure 3, the room type in the room table was set to 'VARCHAR()' when it was more reasonable to assign it to 'ENUM' as there are only a fixed set of values. A single room, a double room, and a suite. However, 'ENUM' was used for the room status. Before, I had also set the payment amount in figure 5 to 'INT' when it was more suitable to set it to 'DECIMAL.' This ensures accurate storage of currency with two decimal places to avoid rounding errors. A great data type to use for monetary values.

The second challenge I had to face was a foreign key constraint error. In figure 4, I forgot to declare the customer and room ID as columns before referencing them as foreign keys for my reservation table. There were also logical inconsistencies as well that I had to change in my sample data. For instance, room 105 in figure 9 was set to maintenance and in the reservation table in figure 11, it was set to reserved. This did not make sense since a room cannot be reserved if a room is under maintenance. The reservation status was then changed to cancelled.

I often would make alias errors in my queries. For instance, the 'JOIN' condition had "C.customer_id = C.customer_id" instead of "C.customer_id = R.customer_id." See the query in figure 16. Lastly, I realised that deleting a customer with existing reservations would cause a foreign key error. To fix this I would have to first delete from Payment 5, then delete Reservation 5 then, Customer 5.

Conclusion

In this project, I have learned thoroughly how to create a hotel reservation system using MySQL. During the process, I have countered many challenges with data type decisions, foreign key constraint errors, logical inconsistencies and alias errors in my queries. However, I also learned to execute queries using real business logic in a hotel reservation system. From finding the total payment to finding the total revenue of a room type. I have also learned what and when to effectively use the different data types. For instance, when to use 'ENUM' instead of 'VARCHAR()' or when to use 'DECIMAL' instead of 'INT.' In the future, the system could be expanded to include additional room types for example to improve the hotel reservation experience.